



TEMA: PROLOG

1.- OBJETIVOS

Al finalizar la práctica el estudiante estará en condiciones de:

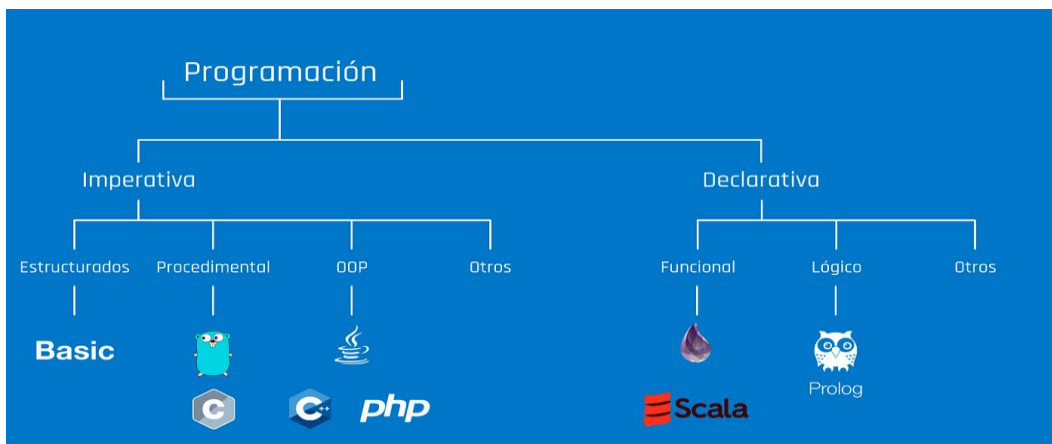
- Implementar algoritmos con el paradigma declarativo en el lenguaje de programación PROLOG.
- Utilizar la herramienta SWI Prolog.

2.- TRABAJO PREPARATORIO

- Descargar e instalar el software SWI Prolog.
- Estudiar los conceptos básicos del lenguaje de programación PROLOG.

3.- BASE TEORICA COMPLEMENTARIA

Paradigmas de lenguajes de programación



La programación declarativa es un paradigma de programación en el que se describe qué se desea obtener como resultado, sin especificar los pasos o procedimientos para alcanzar ese resultado.

Lenguaje de programación PROLOG

PROLOG usa Lógica de Predicados de Primer Orden (restringida a cláusulas de Horn) para representar datos y conocimiento, utiliza encadenamiento hacia atrás y una estrategia de control retroactiva sin información heurística (backtracking).

Las características más relevantes de PROLOG son :

Declarativo: Prolog es un lenguaje declarativo en el que se especifica qué se quiere lograr en lugar de cómo hacerlo. Los programas Prolog se basan en reglas y hechos que representan relaciones y predicados, permitiendo al programador centrarse en la lógica y las relaciones entre los datos.

Programación basada en reglas: Prolog utiliza reglas lógicas (cláusulas de Horn) para representar conocimientos y relaciones. Las reglas se definen mediante cláusulas de la forma "cabeza :- cuerpo", donde la cabeza establece una relación y el cuerpo especifica las condiciones que deben cumplirse para que la relación sea verdadera.

Unificación: La unificación es un proceso fundamental en Prolog. Permite combinar términos y variables para establecer relaciones entre ellos. La unificación se utiliza para resolver consultas y encontrar soluciones a través de la inferencia lógica.

Resolución de consultas: En Prolog, se pueden realizar consultas interactivas al programa, lo que permite obtener respuestas a partir de las reglas y hechos definidos. El sistema de inferencia de Prolog utiliza la unificación y la resolución para buscar soluciones a las consultas, explorando el espacio de búsqueda de manera eficiente.

Recursión: Prolog es un lenguaje recursivo por naturaleza. La recursión permite la definición de reglas que se llaman a sí mismas, lo que facilita la solución de problemas que requieren una exploración o búsqueda en profundidad.

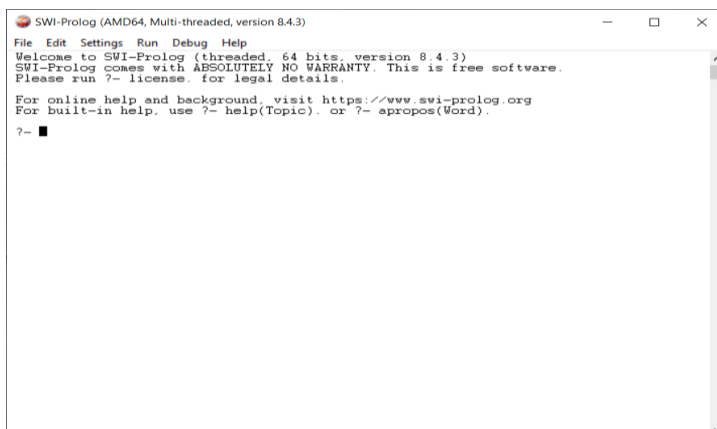
Orientado a patrones: En Prolog, los patrones se utilizan para especificar las estructuras y relaciones de los datos. Esto permite realizar coincidencias y descomponer términos en subestructuras más pequeñas, facilitando la manipulación y el procesamiento de los datos.

Manipulación de listas: Prolog tiene un soporte integrado para la manipulación de listas. Las listas se utilizan ampliamente para representar datos estructurados, y Prolog proporciona operaciones y predicados para trabajar con ellas, como agregar elementos, eliminar elementos, concatenar listas, entre otros.

4.- CONTENIDO DE LA PRÁCTICA.

PARTE 1.- Entorno de trabajo SWI Prolog

- Ejecutar el programa SWI Prolog. Se mostrará la siguiente pantalla:



Al contrario que la mayoría de los lenguajes de programación, Prolog es un lenguaje conversacional; es decir, el sistema Prolog mantiene un diálogo continuo con el programador desde el inicio de la sesión hasta el final de esta. Este diálogo toma generalmente la forma de un interrogatorio, a lo largo del cual el programador planteará preguntas al sistema Prolog. Por su parte, el sistema Prolog responderá cada una de las preguntas formuladas por el programador en la medida en que esto sea posible.

Prolog le indica al programador que está esperando a que éste le formule una pregunta mostrando en pantalla el siguiente símbolo

?-

Tras este símbolo, el programador puede teclear una pregunta (*terminada en un punto*) y pulsar el retorno de carro. Con ello, el programador solicita al sistema Prolog que responda a la pregunta recién formulada. Una vez procesada la pregunta el sistema Prolog mostrará en pantalla la respuesta correspondiente.

Configuración básica de directorios y archivos de programas prolog

- Ejecutar el siguiente comando:

```
pwd.
```

Se mostrará el directorio actual (la ruta y la carpeta) con la que interactúa por defecto el SWI Prolog.

```
?- pwd.  
% c:/users/rozas acurio/documents/prolog/  
true.  
?- ■
```

- Ejecutar el comando **cd** si se desea cambiar de directorio, tal la siguiente figura:

```
?- cd('E:/Practica/swiprolog/programas').  
true.  
?- ■
```

Notar que SWI Prolog utiliza el símbolo “/” en lugar del símbolo “\” para separar los directorios en la especificación de la ruta.

- Ejecutar el comando **ls** para ver el contenido (lista de subdirectorios y archivos) del directorio actual. Se debe mostrar una figura similar a la siguiente:

```
?- ls.  
% animales.pl          mayor.pl~          recursividad.pl  
% animales01.pl       promedio_notas.pl  recursividad01.pl  
% criminal.pl         promedio_notas01.pl saludo.pl  
% mayor.pl            promedio_notas02.pl sistemas expertos/  
true.  
?- ■
```

PARTE 2- Programación en Prolog – Conceptos y ejercicios preliminares

Identificadores

- Los nombres de predicados y las constantes del dominio empiezan con minúscula
- Literales numéricos, como en cualquier lenguaje de programación.
- Cadenas de caracteres entre comillas simples.
- Las variables empiezan con mayúsculas, pueden estar conformados por: Cadenas de letras, dígitos y el símbolo `_`.

- Los comentarios se ponen entre los símbolos `/*` y `*/`. También se puede utilizar el símbolo `%` para un comentario de una sola línea.

Hechos

Se representan como átomos con Lógica de Predicados, empiezan con minúscula y terminan con un punto.

escuelaAcreditada('Ingeniería Informática y de Sistemas').

Reglas

También se representan con Lógica de predicados, donde, la implicación se representa con `:-`. A la izquierda se pone la cabeza (Consecuente) y a la derecha el cuerpo (antecedente).

escuelasCalidad(X) :- escuelaAcreditada(X).

Programas Prolog

Para escribir programas Prolog seleccionar la opción **File** del menú principal, luego la opción **new**. Se debe escribir el nombre del archivo. Se abrirá una ventana con el editor para escribir el nuevo programa, tal la siguiente figura:



Estructura secuencial: Input/output y uso de variables

Ejercicio 1.- Programa de saludo.

Este ejercicio ejemplifica el uso de las sentencias Input/Output en Prolog, así como, el uso de variables.

- Digite el siguiente código:

```
distancia_puntos.pl [modified] promedio_notas.pl saludo.pl
saludo :- write('Hola soy un sistema experto. ¿Cómo te llamas? '),nl,
         read(Nombre),nl,
         write('mucho gusto '),
         write(Nombre).
```

- Seleccionar la opción **File, Save buffer** para almacenar el programa.
- Seleccionar la opción **Compile, Compile Buffer** para compilar el programa.
- Para ejecutar el programa utilice la pantalla principal el SWI Prolog. Digite tal la siguiente figura:

```
true.
?- saludo().
Hola soy un sistema experto. ¿Cómo te llamas?
|: █
```

Ingresar el nombre (en minúscula). Se mostrará

```

?- saludo().
Hola soy un sistema experto. ¿Cómo te llamas?
|: arturo.

mucho gusto arturo
true.
?- ■

```

Ejercicio 2.- Programa para calcular el promedio de tres notas.

- Digite el siguiente código:

```

promedio_notas.pl [modified] promedio_notas.pl
promedio_notas :- write('Cálculo del promedio de tres notas.'), nl,
                  write('Ingrese Nota1: '),
                  read(Nota1), nl,
                  write('Ingrese Nota2: '),
                  read(Nota2), nl,
                  write('Ingrese Nota3: '),
                  read(Nota3), nl,
                  P is (Nota1 + Nota2 + Nota3)/3,
                  write('Promedio = '),
                  write(P), nl.

```

- Almacenar, compilar y ejecutar, tal como se explicó en el ejercicio anterior. Se debe mostrar la siguiente figura.

```

?- promedio_notas.
Cálculo del promedio de tres notas.
Ingrese Nota1: 15.
Ingrese Nota2: |: 16.
Ingrese Nota3: |: 14.
Promedio = 15
true.
?- ■

```

Ejercicio 3.- Implementar un programa que calcule la distancia entre dos puntos en el plano.

Estructura condicional:

Ejercicio 4.- Programa para determinar el mayor de tres números.

- Digite el siguiente código:

```

mayor3.pl
/*Programa para calcular el mayor de tres números */
leer_nros(N1, N2, N3) :-
    write('Ingrese nro 1: '),
    read(N1), nl,
    write('Ingrese nro 2: '),
    read(N2), nl,
    write('Ingrese nro 3: '),
    read(N3), nl.

mayor_tres(N1, N2, N3, M) :- (N1 >= N2, N1 >= N3, M = N1);
                             (N2 >= N1, N2 >= N3, M = N2);
                             (N3 >= N1, N3 >= N2, M = N3).

mostrar_mayor(M) :-
    write('Mayor = '), write(M), nl.

mayor :-
    leer_nros(N1, N2, N3),
    mayor_tres(N1, N2, N3, M),
    mostrar_mayor(M).

```

- Almacenar, compilar y ejecutar el programa. Se mostrará la siguiente figura:

```

true.
?- mayor.
Ingrese nro 1: 26.
Ingrese nro 2: |: 45.
Ingrese nro 3: |: 32.
Mayor = 45
true.
?- ■

```

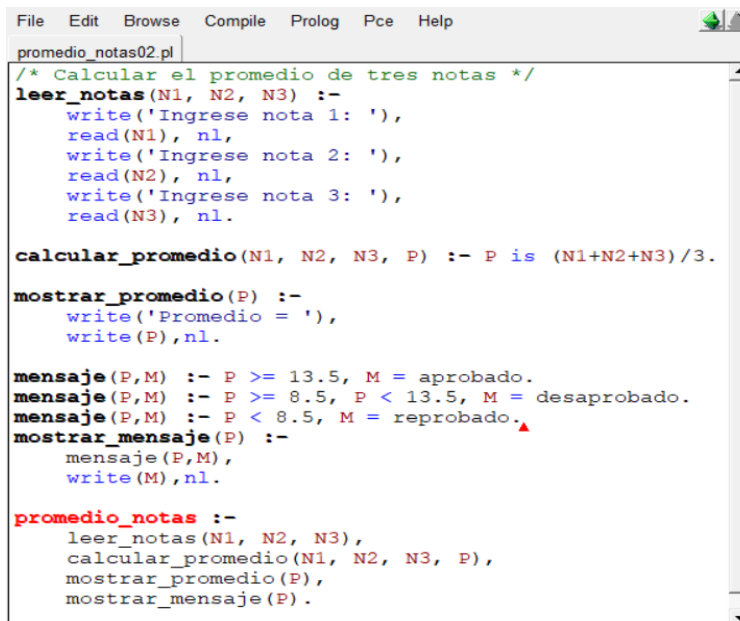
Ejercicio 5.- Implementar un programa que determine si un número de un dígito es o no primo.

Estructura repetitiva:

El motor de inferencia del Prolog, ejecuta automáticamente de manera repetitiva las sentencias que tienen el mismo nombre, hasta que se satisfaga una de las entencias. En el caso del siguiente ejemplo, el predicado *mensaje* se ejecuta de modo repetitivo hasta que se satisfaga uno de ellos.

Ejercicio 6.- Programa para calcular el promedio de tres notas y mostrar si corresponde a aprobado, desaprobado o reprobado.

- Digite el siguiente código:



```

File Edit Browse Compile Prolog Pce Help
promedio_notas02.pl
/* Calcular el promedio de tres notas */
leer_notas(N1, N2, N3) :-
    write('Ingrese nota 1: '),
    read(N1), nl,
    write('Ingrese nota 2: '),
    read(N2), nl,
    write('Ingrese nota 3: '),
    read(N3), nl.

calcular_promedio(N1, N2, N3, P) :- P is (N1+N2+N3)/3.

mostrar_promedio(P) :-
    write('Promedio = '),
    write(P), nl.

mensaje(P,M) :- P >= 13.5, M = aprobado.
mensaje(P,M) :- P >= 8.5, P < 13.5, M = desaprobado.
mensaje(P,M) :- P < 8.5, M = reprobado.
mostrar_mensaje(P) :-
    mensaje(P,M),
    write(M), nl.

promedio_notas :-
    leer_notas(N1, N2, N3),
    calcular_promedio(N1, N2, N3, P),
    mostrar_promedio(P),
    mostrar_mensaje(P).

```

- Almacenar, compilar y ejecutar el programa. Se mostrará la siguiente figura:

```

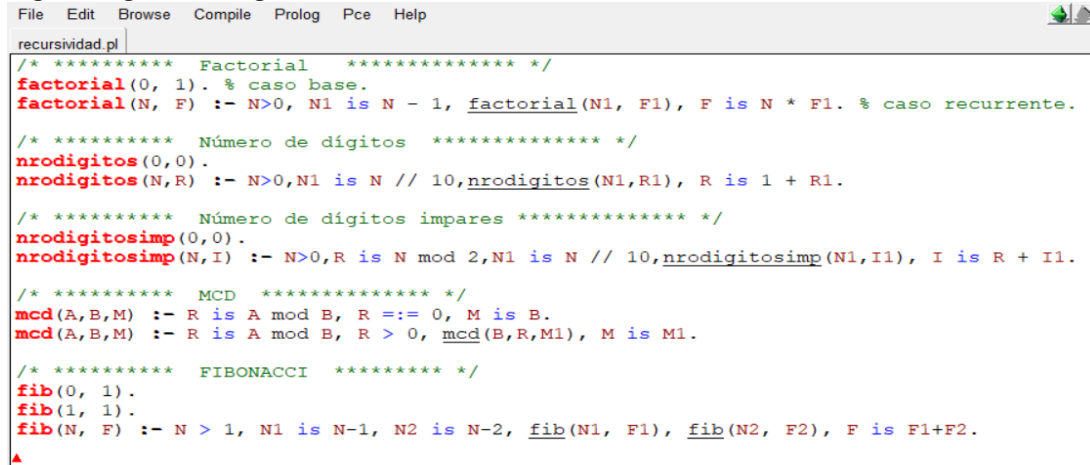
?- promedio_notas.
Ingrese nota 1: 15.
Ingrese nota 2: |: 16.
Ingrese nota 3: |: 14.
Promedio = 15
aprobado
true.
?- ■

```

Recursividad:

Ejercicio 5.- Programa para calcular algunos algoritmos recursivos fundamentales, como factorial, máximo común divisor, fibonacci, etc.

- Digite el siguiente código:



```
File Edit Browse Compile Prolog Pce Help
recursividad.pl
/* ***** Factorial ***** */
factorial(0, 1). % caso base.
factorial(N, F) :- N>0, N1 is N - 1, factorial(N1, F1), F is N * F1. % caso recurrente.

/* ***** Número de dígitos ***** */
nrodigitos(0,0).
nrodigitos(N,R) :- N>0, N1 is N // 10, nrodigitos(N1,R1), R is 1 + R1.

/* ***** Número de dígitos impares ***** */
nrodigitosimp(0,0).
nrodigitosimp(N,I) :- N>0, R is N mod 2, N1 is N // 10, nrodigitosimp(N1,I1), I is R + I1.

/* ***** MCD ***** */
mcd(A,B,M) :- R is A mod B, R =:= 0, M is B.
mcd(A,B,M) :- R is A mod B, R > 0, mcd(B,R,M1), M is M1.

/* ***** FIBONACCI ***** */
fib(0, 1).
fib(1, 1).
fib(N, F) :- N > 1, N1 is N-1, N2 is N-2, fib(N1, F1), fib(N2, F2), F is F1+F2.
^
```

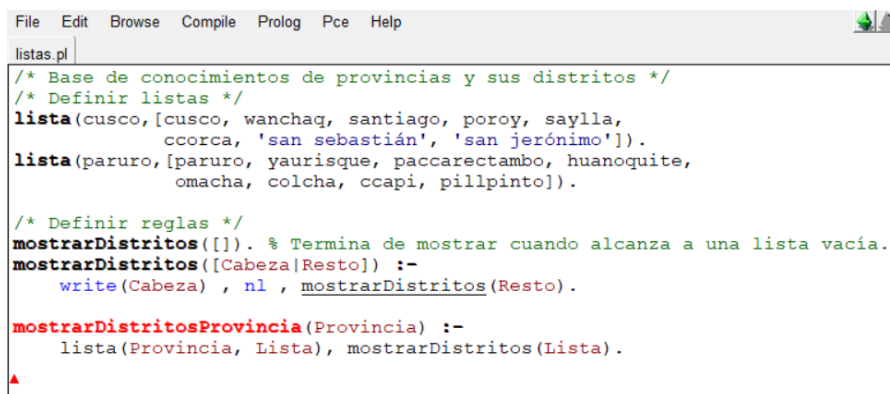
- Almacenar, compilar y ejecutar cada algoritmo. Por ejemplo, para ejecutar factorial, ejecutar tal como se muestra en la figura:

```
true.
?- factorial(5,F).
F = 120 ^
```

Estructura de datos - Listas

Ejercicio 6.- Programa para gestionar provincias y sus distritos.

- Digite el siguiente código:



```
File Edit Browse Compile Prolog Pce Help
listas.pl
/* Base de conocimientos de provincias y sus distritos */
/* Definir listas */
lista(cusco,[cusco, wanchaq, santiago, poroy, saylla,
             ccorca, 'san sebastián', 'san jerónimo']).
lista(paruro,[paruro, yaurisque, paccarectambo, huanquite,
             omacha, colcha, ccapi, pillpinto]).

/* Definir reglas */
mostrarDistritos([]). % Termina de mostrar cuando alcanza a una lista vacía.
mostrarDistritos([Cabeza|Resto]) :-
    write(Cabeza) , nl , mostrarDistritos(Resto).

mostrarDistritosProvincia(Provincia) :-
    lista(Provincia, Lista), mostrarDistritos(Lista).
^
```

- Almacenar, compilar y ejecutar cada algoritmo. Por ejemplo, para mostrar los distritos de la ciudad del cusco, ejecutar tal como se muestra en la figura:

```
?- mostrarDistritosProvincia(cusco).  
cusco  
wanchaq  
santiago  
poroy  
saylla  
ccorca  
san sebastián  
san jerónimo  
false.  
?- ■
```

3.- EJERCICIOS COMPLEMENTARIOS.

1. Implementar algoritmos recursivos para:
 - Las torres de Hanoi.
 - En una base Militar en el desierto hay un Jeep y n bidones de gasolina. El tanque de gasolina del Jeep tiene la capacidad para un bidón y además puede cargar otro bidón. El Jeep puede recorrer D Km con un bidón de gasolina. Escribir un subprograma que calcule la distancia máxima que puede recorrer el Jeep.
2. Investigar las diferentes operaciones que se pueden efectuar con listas. Poner un ejemplo para cada tipo de operación.

Nota.- Presentar la solución de estos ejercicios al finalizar la sesión de laboratorio.